SentinelIQ 11 — Test & Deployment Plan

Oct 13, 2026 · @maxi

Every change must pass automated tests and a containerized build before it can reach production, and deployment is automatic only from the main branch.

Test layers

Layer	What it checks	Tool	Runs
Unit	Normalization, feature building, threshold gate, output validation	pytest	Every commit, locally and in CI
Integration	API routes against a real PostgreSQL container: auth, ingestion, idempotency, audit rules	pytest with an HTTP test client, Postgres in Docker	Every pull request
Contract	Responses match the OpenAPI document from Document 08	Schema-based testing, for example Schemathesis	Every pull request
Frontend component	Dashboard components and their loading, empty and error states	Vitest with React Testing Library	Every pull request
End to end	Login to recorded decision in a browser against the full Docker stack	Playwright	Pull requests to main and before release
Security	Dependency audit, container image scan, secret scan, adversarial prompt set	pip-audit and npm audit, Trivy, gitleaks	Every pull request
Load	Ingestion latency and queue behaviour against the targets in Document 03	Locust or k6	Phase 7 and before release

Model and retrieval regression tests

Quality is tested like code, so a change that makes the model or the explanations worse fails the build.

- **Model floor:** a CI job scores a fixed held-out sample and fails if recall or false-positive rate is worse than a floor set just below the Phase 4 baseline.
- **Retrieval check:** for each hand-labeled event description, the expected ATT&CK technique must appear in the top results; the build fails if the hit rate drops.
- **Injection suite:** a set of malicious log lines must not change the validated output.
- **Deterministic LLM calls in CI:** tests use recorded responses, so they cost nothing and give the same result every time. A separate scheduled job runs the same checks against the live provider.

CI pipeline

GitHub Actions runs these stages in order on every pull request; a failure at any stage blocks the merge.

1. **Trigger:** a pull request, or a push to the main branch.
2. **Lint and type checks:** Python and frontend.
3. **Unit tests.**
4. **Integration and contract tests:** a PostgreSQL service starts, migrations run, then the API tests execute.
5. **Model and retrieval regression tests:** using recorded LLM responses.
6. **Build and scan:** container images are built, then scanned together with the dependencies, and the repository is scanned for secrets.
7. **Frontend and end-to-end tests:** end-to-end runs on pull requests to main.
8. **Merge:** allowed only when every check is green.
9. **Release (from Phase 7):** on main, images are pushed and deployed to Cloud Run.

Environments

Document 07 describes how each environment runs; this table shows what each is used to verify.

Environment	Used for	Secrets and data
Local	Development and the demo	Local environment file; seed data
CI	Automated checks on every pull request	No real secrets; fixtures and recorded LLM responses

Environment	Used for	Secrets and data
Production	The public demo link (Phase 7)	Secrets in a secret manager; public and synthetic data only

Deployment and rollback

- **Packaging:** each service is built into a container image tagged with the commit hash.
- **Release trigger:** a merge to main that passes every check; production deploys happen from Phase 7 onward.
- **Order of steps:** database migrations run first and must be backward compatible with the previous release, then the new revision is deployed.
- **Gradual rollout:** Cloud Run keeps earlier revisions, so traffic can move to the new revision after its health check passes.
- **Rollback:** send traffic back to the previous revision. Because migrations are backward compatible, no data change has to be undone.
- **Smoke test:** after each deployment, a short script logs in, sends one event and confirms that it appears in the queue.

Observability

- **Logs:** structured JSON that carries the request ID, job ID and model version. Secrets and full raw event payloads are never logged (threat T7).
- **Health checks:** a liveness route that shows the process is up, and a readiness route that confirms the database can be reached.
- **Metrics to collect:** ingestion rate, request latency, queue depth, age of the oldest job, scoring and explanation durations, explanation failure rate, and LLM call count and errors.
- **Alerts:** the oldest job waiting too long, a high explanation failure rate, or failing readiness. Thresholds are set from the Phase 7 measurements, not guessed now.
- **Where it shows:** the cloud platform's built-in monitoring, plus the in-product metrics view from FR8.